

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)
Weightage:	30%	Total Marks:	25
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	G. N. Mogana Priya	Reg.No.:	192321165
Department:	IT	Date:	

ANNEXURE A
Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Code of Conduct:

I MoganaPriya (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.


Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)
Q1	2	2.5	1.5	1	1
Q2	2	2.5	1.5	1	1
Q3	1.5	2	2	0.5	0.5
Q4	1.5	3	2	0.5	0.5
Q5	2	3	1.5	1	1

RUBRICS FOR CALCULATION:

38.5

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20% 2	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30% 3	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20% 2	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10% 1	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20% 2	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. DMA-Based Data Transfer Algorithm

Purpose

To design a DMA-based data transfer algorithm that transfers data from an I/O device to memory while minimizing CPU involvement and using interrupts to indicate the completion of the transfer.

Algorithm

ALGORITHM DMA_Data_Transfer

BEGIN

 Initialize DMA_Controller

 IF IO_Device is NOT_READY THEN

 Transfer_Status $\leftarrow$ "Device Not Ready"

 RETURN Transfer_Status

 END IF

 DMA_Source $\leftarrow$ IO_Device

 DMA_Destination $\leftarrow$ Memory_Address

 DMA_Count $\leftarrow$ Data_Size

 Enable DMA_Interrupt

 Start DMA_Transfer

 WHILE DMA_Transfer is ACTIVE DO

 CPU performs other tasks

 END WHILE

 WHEN DMA_Interrupt occurs DO

 Read DMA_Status

IF DMA_Status = COMPLETE THEN

Acknowledge Interrupt

Transfer_Status $\leftarrow$ "Transfer Successful"

ELSE

Acknowledge Interrupt

Handle DMA_Error

Transfer_Status $\leftarrow$ "Transfer Failed"

END IF

END WHEN

RETURN Transfer_Status

END

Required Parameters

- I/O device
- Starting memory address
- Data size
- DMA controller

Expected Result

- Data is transferred from the I/O device to memory.
- Transfer completion or failure status is generated.

Design Considerations

- CPU involvement must be minimized.
- DMA must perform the actual data transfer.
- An interrupt must notify the CPU after completion.
- Device-not-ready and transfer-error conditions must be handled.

Summary

The DMA controller is initialized with the source, destination, and data size. It transfers data directly to memory while the CPU performs other tasks. An interrupt notifies the CPU when the operation is completed or an error occurs.

2. Vectored Interrupt Handling System

Purpose

To design a vectored interrupt handling algorithm that prioritizes interrupt requests from multiple devices and dispatches the appropriate Interrupt Service Routine (ISR).

Algorithm

ALGORITHM Vectored_Interrupt_Handler

BEGIN

Initialize Interrupt_Controller

Load Interrupt_Vector_Table

WHILE System is RUNNING DO

IF Interrupt_Request exists THEN

Pending_Interrupts $\leftarrow$ Get_Pending_Interrupts()

Highest_Priority_Interrupt $\leftarrow$

Select_Highest_Priority(

Pending_Interrupts,

Device_Priorities

)

Save CPU_Context

Acknowledge Highest_Priority_Interrupt

Vector_Address $\leftarrow$

Interrupt_Vector_Table[

Highest_Priority_Interrupt

]

Execute ISR at Vector_Address

IF ISR completes successfully THEN

Clear Interrupt_Flag

Send End_Of_Interrupt

Interrupt_Status $\leftarrow$ "Successful"

ELSE

Handle ISR_Error

Interrupt_Status $\leftarrow$ "Failed"

END IF

Restore CPU_Context

ELSE

CPU continues normal execution

END IF

END WHILE

END

Required Parameters

- Interrupt requests
- Device priority levels

- Interrupt vector table
- Interrupt controller

Expected Result

- Highest-priority interrupt is serviced first.
- Correct ISR is executed.
- Interrupt service status is generated.

Design Considerations

- Multiple simultaneous interrupts must be supported.
- Higher-priority interrupts must be serviced first.
- The correct ISR must be selected using the vector table.
- CPU context must be preserved and restored.
- Interrupt errors must be handled.

Operational Summary

The interrupt controller monitors requests and identifies pending interrupts. The highest-priority interrupt is selected, its vector address is obtained, and the corresponding ISR is executed. After servicing, the CPU restores its previous context and resumes normal operation.

3. Dynamic I/O Mechanism Selection

Purpose

To design an algorithm that dynamically selects between memory-mapped I/O and I/O-mapped I/O based on latency, bandwidth, and processor support.

Algorithm

ALGORITHM Select_IO_Mechanism

BEGIN

IF CPU_Support = MEMORY_MAPPED_ONLY THEN

Selected_IO_Mode $\leftarrow$ MEMORY_MAPPED_IO

ELSE IF CPU_Support = IO_MAPPED_ONLY THEN

Selected_IO_Mode $\leftarrow$ IO_MAPPED_IO

ELSE

IF Latency_Requirement = HIGH AND

Bandwidth_Requirement = HIGH THEN

Selected_IO_Mode $\leftarrow$ MEMORY_MAPPED_IO

ELSE IF Latency_Requirement = HIGH THEN

Selected_IO_Mode $\leftarrow$ MEMORY_MAPPED_IO

ELSE IF Bandwidth_Requirement = HIGH THEN

Selected_IO_Mode $\leftarrow$ MEMORY_MAPPED_IO

ELSE

Selected_IO_Mode $\leftarrow$ IO_MAPPED_IO

END IF

END IF

Configure_IO_Controller(Selected_IO_Mode)

RETURN Selected_IO_Mode

END

Required Parameters

- Latency requirement
- Bandwidth requirement
- CPU support
- I/O controller

Expected Result

- Appropriate I/O mechanism is selected.
- I/O controller is configured according to the selected mode.

Design Considerations

- CPU support for the I/O mechanisms must be checked.
- Latency requirements must be considered.
- Bandwidth requirements must be considered.
- The selected I/O mechanism must be correctly configured.

Operational Summary

The algorithm first checks the available I/O mechanisms supported by the CPU. It then evaluates latency and bandwidth requirements and selects the appropriate I/O mode before configuring the I/O controller.

4. Multi-Interface Bus Arbitration

Purpose

To design a bus arbitration algorithm that manages communication requests from PCIe, USB, and Thunderbolt interfaces while maintaining priority and fair resource allocation.

Algorithm

ALGORITHM Multi_Interface_Bus_Arbitration

BEGIN

Initialize Arbitration_Controller

WHILE System is RUNNING DO

```
Request_List ← Collect_All_Requests(  
    PCIe_Requests,  
    USB_Requests,  
    Thunderbolt_Requests  
)
```

```
IF Request_List is EMPTY THEN
```

```
    Continue Monitoring
```

```
ELSE
```

```
    FOR each Request in Request_List DO
```

```
        Assign Priority(  
            Request,  
            Device_Priorities  
        )
```

```
    END FOR
```

```
Highest_Priority_Request ←  
    Select_Highest_Priority(Request_List)
```

```
IF Multiple_Requests_Have_Same_Priority THEN
```

```
    Selected_Request ←  
        Apply_Round_Robin(Request_List)
```

```
ELSE
```

```
    Selected_Request ←  
        Highest_Priority_Request
```

END IF

Grant_Access(Selected_Request)

WHILE Data_Transfer is ACTIVE DO

IF Transfer_Error occurs THEN

Handle Transfer_Error

Revoke_Access(Selected_Request)

BREAK

END IF

END WHILE

Release_Access(Selected_Request)

Granted_Device $\leftarrow$ Selected_Request

END IF

END WHILE

END

Required Parameters

- PCIe requests
- USB requests
- Thunderbolt requests
- Device priority levels

- Arbitration controller

Expected Result

- One requesting device is granted communication access.
- Data transfer is completed or an error is handled.
- Communication resources are released after use.

Design Considerations

- Multiple requests must be handled.
- Higher-priority requests should be serviced first.
- Equal-priority requests should be handled fairly.
- Transfer errors must be detected.
- A device must release the resource after completing communication.

Operational Summary

The arbitration controller collects requests from PCIe, USB, and Thunderbolt interfaces. It assigns priorities and grants access to the highest-priority request. Round-robin scheduling is used for equal-priority requests to provide fairness and avoid starvation.

5. Hybrid I/O Communication Mechanism

Purpose

To design a hybrid I/O communication algorithm that uses polling for low-speed devices and DMA with interrupts for high-speed devices to improve transfer efficiency and reduce CPU overhead.

Algorithm

ALGORITHM Hybrid_IO_Communication

BEGIN

IF Device_Speed = LOW_SPEED THEN

 Initialize Device

 REPEAT

 Read Device_Status

 UNTIL Device_Status = READY

WHILE Data_Remains DO

 Data $\leftarrow$ Read_From_Device()

 Store Data in Memory_Buffer

END WHILE

Transfer_Status $\leftarrow$

 "Polling Transfer Successful"

ELSE

 Initialize DMA_Controller

 DMA_Source $\leftarrow$ Device

 DMA_Destination $\leftarrow$ Memory_Buffer

 DMA_Count $\leftarrow$ Data_Size

 Enable DMA_Interrupt

 Start DMA_Transfer

 WHILE DMA_Transfer is ACTIVE DO

 CPU performs other tasks

 END WHILE

 WHEN DMA_Interrupt occurs DO

 IF DMA_Status = COMPLETE THEN

 Acknowledge Interrupt

```
Transfer_Status ←  
    "DMA Transfer Successful"
```

```
ELSE
```

```
    Handle DMA_Error  
    Transfer_Status ←  
        "DMA Transfer Failed"
```

```
END IF
```

```
END WHEN
```

```
END IF
```

```
RETURN Transfer_Status
```

```
END
```

Required Parameters

- I/O device
- Device speed
- Data size
- Memory buffer
- DMA controller

Expected Result

- Low-speed devices are serviced using polling.
- High-speed devices are serviced using DMA and interrupts.
- Data is transferred successfully or an error status is generated.

Design Considerations

- Low-speed devices must use polling.
- High-speed devices must use DMA.

- DMA completion must generate an interrupt.
- CPU involvement must be minimized during high-speed transfers.
- Transfer errors must be handled.

Operational Summary

The algorithm determines the device speed and selects the appropriate communication method. Polling is used for low-speed devices, while DMA with interrupts is used for high-speed devices. This hybrid approach provides efficient communication while reducing unnecessary CPU involvement.